目 录

CONTENTS

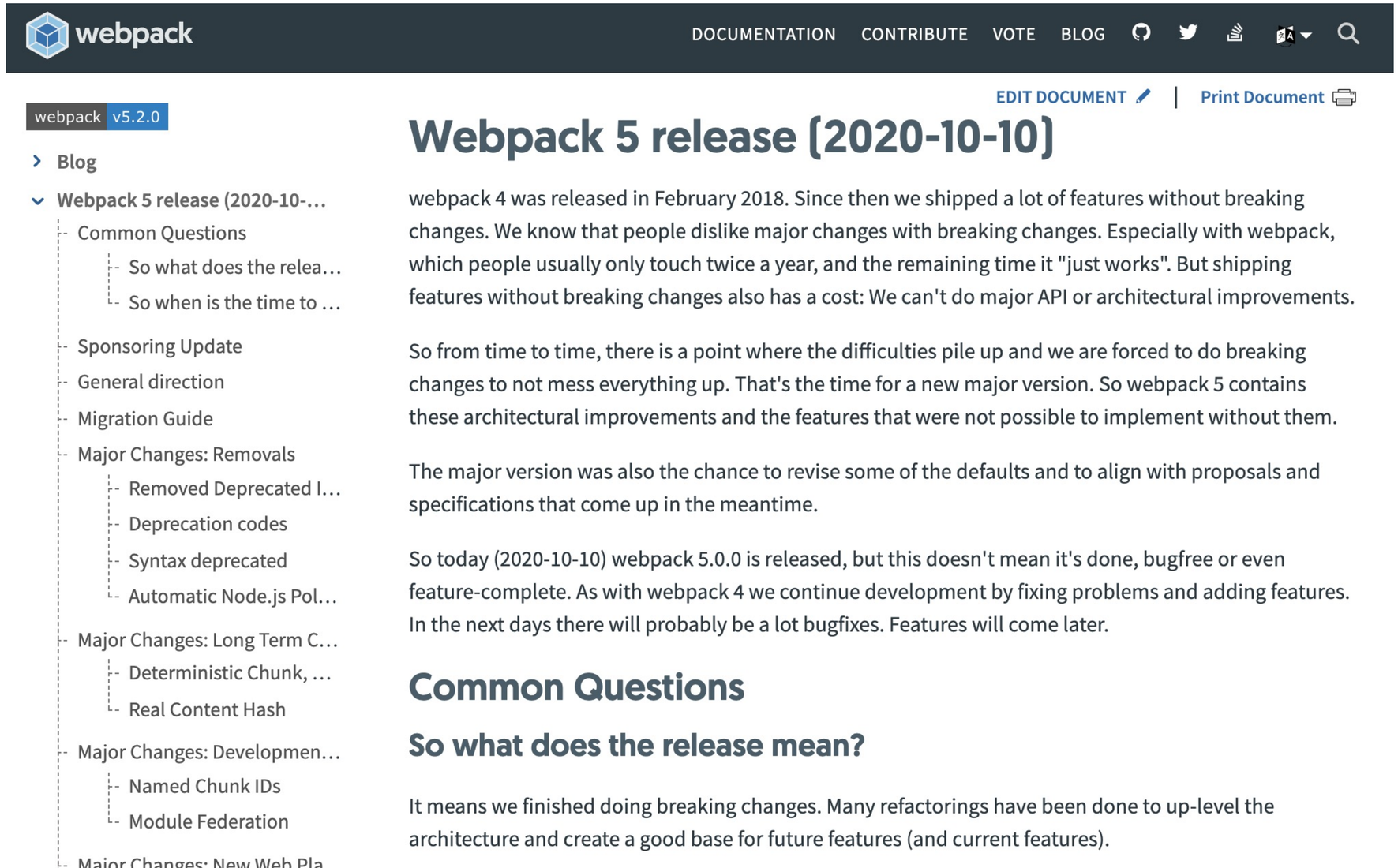
01 | webpack 5 新特性解析

02 | bundle 和 bundless 的差异

03 | Vite 的构建原理

webpack 5 在 2020–10–10 正式发布

官方 changelog：https://webpack.js.org/blog/2020-10-10-webpack-5-release/



webpack 5 新特性概览



最小 Node.js 版本：10.13.0

最低支持的 Node.js 版本从 6 增加到 10.13.0



功能清除：清理弃用的能力

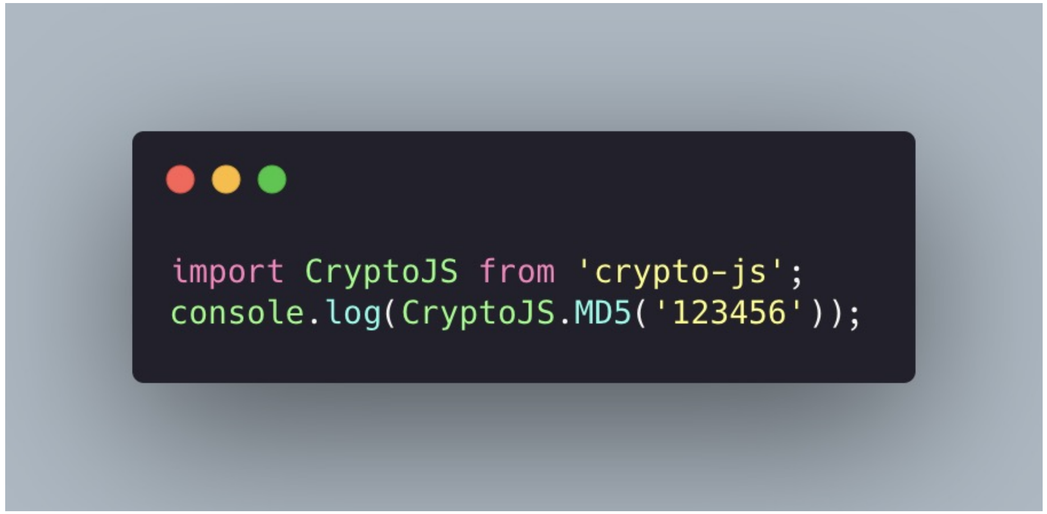
所有在 webpack 4 里面被废弃的能力都被清除，因此需要确保 webpack4 没有打印警告。

require.includes 语法已被废弃

- 可以通过 Rule.parser.requireInclude 将行为改为允许、废弃或禁用。

功能清除：不再为 Node.js 模块引入 polyfill

之前的版本是：如果某个模块依赖 Node.js 里面的核心模块，那么这个模块被引入的时候会把 Node.js 整个 polyfill 顺带引入。



```
import CryptoJS from 'crypto-js';
console.log(CryptoJS.MD5('123456'));
```

```
→  webpack5 npm run build

> my-project@1.0.0 build /Users/bytedance/bytedance/webpack5/nodejs-polyfill/webpack5
> webpack

[webpack-cli] Compilation finished
asset bundle.js 49.7 KiB [emitted] [minimized] (name: main) 1 related asset
runtime modules 878 bytes 4 modules
modules by path ./node_modules/crypto-js/*.js 178 KiB 34 modules
./src/index.js 70 bytes [built] [code generated]
```

webpack 5

```
→  webpack4 npm run build

> my-project@1.0.0 build /Users/bytedance/bytedance/webpack5/nodejs-polyfill/webpack4
> webpack

Hash: 9e46b04344e7a694dee9
Version: webpack 4.44.2
Time: 8464ms
Built at: 11/09/2020 2:44:51 PM
    Asset      Size  Chunks                    Chunk Names
bundle.js  716 KiB       0  [emitted]  [big]  main
```

webpack 4

功能清除：不再为 Node.js 模块引入 polyfill

```
"dependencies": {
  "@webassemblyjs/ast": "1.9.0",
  "@webassemblyjs/helper-module-context": "1.9.0",
  "@webassemblyjs/wasm-edit": "1.9.0",
  "@webassemblyjs/wasm-parser": "1.9.0",
  "acorn": "^6.4.1",
  "ajv": "^6.10.2",
  "ajv-keywords": "^3.4.1",
  "chrome-trace-event": "^1.0.2",
  "enhanced-resolve": "^4.5.0",
  "eslint-scope": "^4.0.3",
  "json-parse-better-errors": "^1.0.2",
  "loader-runner": "^2.4.0",
  "loader-utils": "^1.2.3",
  "memory-fs": "^0.4.1",
  "micromatch": "^3.1.10",
  "mkdirp": "^0.5.3",
  "neo-async": "^2.6.1",
  "node-libs-browser": "^2.2.1",
  "schema-utils": "^1.0.0",
  "tapable": "^1.1.3",
  "terser-webpack-plugin": "^1.4.3",
  "watchpack": "^1.7.4",
  "webpack-sources": "^1.4.1"
}
```

```
exports.assert               = require.resolve('assert/');
exports.buffer               = require.resolve('buffer/');
exports.child_process        = null;
exports.cluster              = null;
exports.console              = require.resolve('console-browserify');
exports.constants            = require.resolve('constants-browserify');
exports.crypto               = require.resolve('crypto-browserify');
exports.dgram                = null;
exports.dns                  = null;
exports.domain               = require.resolve('domain-browser');
exports.events               = require.resolve('events/');
exports.fs                   = null;
exports.http                 = require.resolve('stream-http');
exports.https                = require.resolve('https-browserify');
exports.module               = null;
exports.net                  = null;
exports.os                   = require.resolve('os-browserify/browser.js');
exports.path                 = require.resolve('path-browserify');
exports.punycode             = require.resolve('punycode/');
exports.process              = require.resolve('process/browser.js');
exports.querystring          = require.resolve('querystring-es3/');
exports.readline             = null;
exports.repl                 = null;
exports.stream               = require.resolve('stream-browserify');
exports._stream_duplex       = require.resolve('readable-stream/duplex.js');
exports._stream_passthrough  = require.resolve('readable-stream/passthrough.js');
exports._stream_readable     = require.resolve('readable-stream/readable.js');
exports._stream_transform    = require.resolve('readable-stream/transform.js');
exports._stream_writable     = require.resolve('readable-stream/writable.js');
exports.string_decoder       = require.resolve('string_decoder/');
exports.sys                  = require.resolve('util/util.js');
exports.timers               = require.resolve('timers-browserify');
exports.tls                  = null;
exports.tty                  = require.resolve('tty-browserify');
exports.url                  = require.resolve('url/');
exports.util                 = require.resolve('util/util.js');
exports.vm                   = require.resolve('vm-browserify');
exports.zlib                 = require.resolve('browserify-zlib');
```

node-libs-browser：https://github.com/webpack/node-libs-browser#readme

长期缓存：确定的模块 Id、chunk 和导出名称

在生产模式下，默认的 chunkIds: “deterministic”, moduleIds: “deterministic”。设置成 deterministic 时默认最小 3 位数字会被使用

选项值	描述
`'natural'`	按使用顺序的数字 id。
`'named'`	对调试更友好的可读的 id。
`'deterministic'`	在不同的编译中不变的短数字 id。有益于长期缓存。在生产模式中会默认开启。
`'size'`	专注于让初始下载包大小更小的数字 id。
`'total-size'`	专注于让总下载包大小更小的数字 id。

chunkIds

选荐值	描述
`natural`	按使用顺序的数字 id。
`named`	对调试更友好的可读的 id。
`deterministic`	被哈希转化成的小位数值模块名。
`size`	专注于让初始下载包大小更小的数字 id。

moduleIds

长期缓存：确定的模块 Id、chunk 和导出名称

```
'use strict';

import React from 'react';
import ReactDOM from 'react-dom';

class Search extends React.Component {

    constructor() {
        super(...arguments);

        this.state = {
            Text: null
        };
    }

    loadComponent() {
        import('./text.js').then((Text) => {
            this.setState({
                Text: Text.default
            });
        });
    }

    render() {
        const { Text } = this.state;
        return <div className="search-text">
            搜索文字的内容<img onClick={ this.loadComponent.bind(this) } />
        </div>;
    }
}

ReactDOM.render(
    <Search />,
    document.getElementById('root')
);
```

```
➜  webpack5 npm run build

> my-project@1.0.0 build /Users/bytedance/bytedance/webpack5/deterministic/webpack5
> webpack

[webpack-cli] Compilation finished
asset bundle.js 130 KiB [compared for emit] [minimized] (name: main) 1 related asset
asset 687.bundle.js 220 bytes [compared for emit] [minimized]
runtime modules 6.57 KiB 9 modules
cacheable modules 196 KiB
  modules by path ./node_modules/ 192 KiB
    modules by path ./node_modules/react/ 8.5 KiB 2 modules
    modules by path ./node_modules/react-dom/ 175 KiB
      ./node_modules/react-dom/index.js 1.32 KiB [built] [code generated]
      ./node_modules/react-dom/cjs/react-dom.production.min.js 173 KiB [built] [code
    modules by path ./node_modules/scheduler/ 6.84 KiB
      ./node_modules/scheduler/index.js 197 bytes [built] [code generated]
      ./node_modules/scheduler/cjs/scheduler.production.min.js 6.65 KiB [built] [coc
    ./node_modules/object-assign/index.js 2.17 KiB [built] [code generated]
```

webpack 5

```
➜  webpack4 npm run build

> my-project@1.0.0 build /Users/bytedance/bytedance/webpack5/determinis
> webpack

Hash: eaeb2a094515e1f29151
Version: webpack 4.44.2
Time: 3391ms
Built at: 11/09/2020 2:24:04 PM
      Asset       Size  Chunks             Chunk Names
1.bundle.js  194 bytes       1  [emitted]
  bundle.js    132 KiB       0  [emitted]  main
Entrypoint main = bundle.js
[3] ./src/index.js 3.99 KiB {0} [built]
[8] ./src/text.js 139 bytes {1} [built]
    + 7 hidden modules
```

webpack 4

持久化缓存

在 webpack 4 里面，可以使用 cache-loader 将编译结果写入硬盘缓存，还可以使用 babel-loader，设置 option.cacheDirectory 将 babel-loader 编译的结果写进磁盘。

webpack 5 缓存策略

- 默认开启缓存，缓存默认是在内存里。可以对 cache 进行设置 。
- 缓存淘汰策略：文件缓存存储在 node_modules/.cache/webpack，最大 500MB，缓存时常两个星期，旧的缓存先淘汰。

```
module.exports = {
  cache: {
    // 1. 将缓存类型设置为文件系统
    type: 'filesystem',

    buildDependencies: {
      // 2. 将你的 config 添加为 buildDependency，以便在改变 config 时获得缓存无效
      config: [__filename],

      // 3. 如果你有其他的东西被构建依赖，你可以在这里添加它们
      // 注意，webpack、加载器和所有从你的配置中引用的模块都会被自动添加
    },
  },
};
```

构建优化：Tree Shaking 优化–嵌套的 Tree shaking

```
// inner.js
export const a = 1;
export const b = 2;

// module.js
export * as inner from './inner';
// 或 import * as inner from './inner'; export { inner };

// user.js
import * as module from './module';
console.log(module.inner.a);
```

```
➜  webpack5 cat dist/bundle.js
(()=>{"use strict";console.log(1)})();%
```

webpack 5

```
➜  webpack4 cat dist/bundle.js
!function(e){var r={};function t(n){if(r[n])return r[n].exports;var o=r[n]={i:n,l:!1,exports:{}};return e[n].call(o.exports,o,o
.exports,t),o.l=!0,o.exports}t.m=e,t.c=r,t.d=function(e,r,n){t.o(e,r)||Object.defineProperty(e,r,{enumerable:!0,get:n})},t.r=fu
nction(e){"undefined"!=typeof Symbol&&Symbol.toStringTag&&Object.defineProperty(e,Symbol.toStringTag,{value:"Module"}),Object.d
efineProperty(e,"__esModule",{value:!0})},t.t=function(e,r){if(1&r&&(e=t(e)),8&r)return e;if(4&r&&"object"==typeof e&&e&&e.__es
Module)return e;var n=Object.create(null);if(t.r(n),Object.defineProperty(n,"default",{enumerable:!0,value:e}),2&r&&"string"!=t
ypeof e)for(var o in e)t.d(n,o,function(r){return e[r]}.bind(null,o));return n},t.n=function(e){var r=e&&e.__esModule?function(
){return e.default}:function(){return e};return t.d(r,"a",r),r},t.o=function(e,r){return Object.prototype.hasOwnProperty.call(e
,r)},t.p="",t(t.s=0)}([function(e,r,t){"use strict";t.r(r);var n={};t.r(n),t.d(n,"a",(function(){return o})),t.d(n,"b",(functio
n(){return u}));var o=1,u=2;console.log(n.a)}]);%
```

webpack 4

构建优化：Tree Shaking 优化–内部模块 Tree shaking

```
import { something } from './something';

function usingSomething() {
  return something;
}

export function test() {
  return usingSomething();
}
```

```
➜  webpack5 cat dist/bundle.js
➜  webpack5
```

webpack 5

```
➜  webpack4 cat dist/bundle.js
!function(e){var t={};function r(n){if(t[n])return t[n].exports;var o=t[n]={i:n,l:!1,exports:{}};return
 e[n].call(o.exports,o,o.exports,r),o.l=!0,o.exports}r.m=e,r.c=t,r.d=function(e,t,n){r.o(e,t)||Object.d
efineProperty(e,t,{enumerable:!0,get:n})},r.r=function(e){"undefined"!=typeof Symbol&&Symbol.toStringTa
g&&Object.defineProperty(e,Symbol.toStringTag,{value:"Module"}),Object.defineProperty(e,"__esModule",{v
alue:!0})},r.t=function(e,t){if(1&t&&(e=r(e)),8&t)return e;if(4&t&&"object"==typeof e&&e&&e.__esModule)
return e;var n=Object.create(null);if(r.r(n),Object.defineProperty(n,"default",{enumerable:!0,value:e})
,2&t&&"string"!=typeof e)for(var o in e)r.d(n,o,function(t){return e[t]}.bind(null,o));return n},r.n=fu
```

webpack 4

代码生成：支持生成 ES6 代码

webpack 4 之前只生成 ES5 的代码。 webpack 5 则现在既可以生成 ES5 又可以生成 ES6/ES2015 代码。

两种设置方式：5 =< ecmaVersion <= 11 或 2009 =< ecmaVersion <= 2020



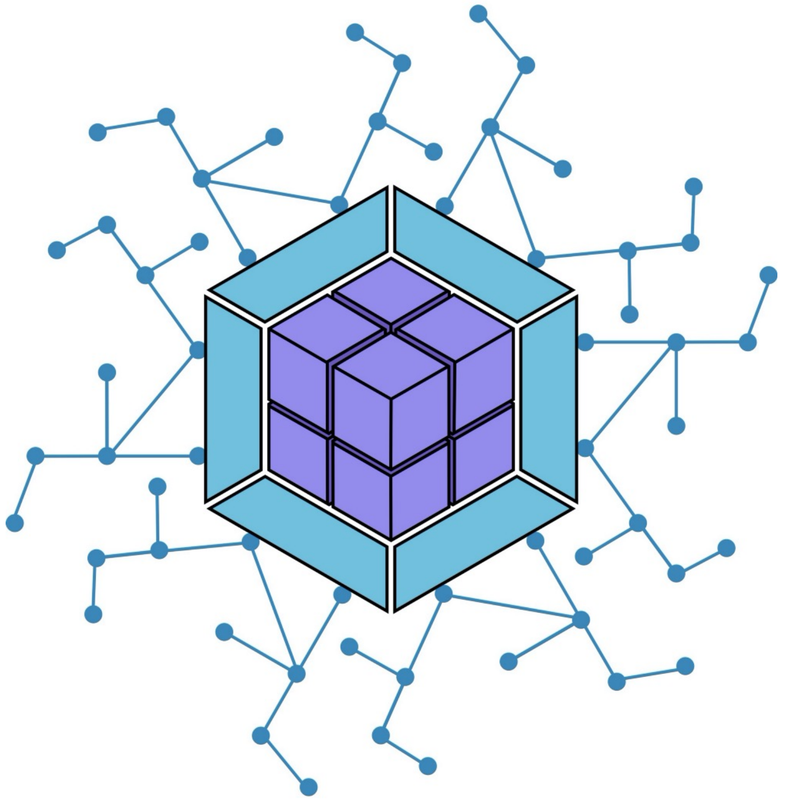
开创性的特性：模块联邦

发明者：Zack Jackson

基本解释：使 一个JavaScript 应用在运行过程中可以动态加载另一个应用的代码，并支持共享依赖（CDN）。不再需要本地安装 npm 包。

- Remote：被依赖方，被 Host 消费的 webpack 构建
- Host：依赖方，消费其他 Remote 的 webpack 构建

一个应用可以是 Host，也可以是 Remote，也可以同时是 Host 和 Remote。



开创性的特性：ModuleFederationPlugin 介绍

Webpack 内部通过 ModuleFederationPlugin 插件将多个应用结合起来。

- name：必须，唯一 ID，作为输出的模块名，使用的时通过 ${name}/${expose} 的方式使用
- library：必须，其中这里的 name 为作为 umd 的 name；
- remotes：可选，表示作为 Host 时，去消费哪些 Remote；
- shared：可选，优先用 Host 的依赖，如果 Host 没有，再用自己的；
- main.js：应用主文件
- remoteEntry.js：作为 remote 时被引的文件

开创性的特性：模块联邦使用案例

```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const { ModuleFederationPlugin } = require("webpack").container;

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: "app1",
      remotes: {
        app2: `app2@${getRemoteEntryUrl(3002)}`,
      },
      shared: { react: { singleton: true }, "react-dom": { singleton: true } },
    }),
    new HtmlWebpackPlugin({
      template: "./public/index.html",
    }),
  ]
};
```

应用一（host）

```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const { ModuleFederationPlugin } = require("webpack").container;

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: "app2",
      library: { type: "var", name: "app2" },
      filename: "remoteEntry.js",
      exposes: {
        "./Button": "./src/Button",
      },
      shared: { react: { singleton: true }, "react-dom": { singleton: true } },
    }),
    new HtmlWebpackPlugin({
      template: "./public/index.html",
    }),
  ]
};
```

应用二（remote）

webpack 5 和 webpack 4 构建速度对比

```
.js??ruleSet[1].rules[2].use[3]!./node_modules/px2rem-loade
dex.js??ruleSet[1].rules[2].use[4]!./src/search2/search.les
0 bytes [built] [code generated]
  modules by path ./node_modules/ 176 KiB
    modules by path ./node_modules/react/ 9.39 KiB 2 module
    modules by path ./node_modules/react-dom/ 155 KiB 2 modu
    modules by path ./node_modules/large-number/ 2.27 KiB 2
ules
    modules by path ./node_modules/scheduler/ 7.37 KiB 2 mo
s
    ./node_modules/object-assign/index.js 2.17 KiB [built]
e generated]
webpack 5.4.0 compiled successfully in 4188 ms
```

```
0 bytes [built] [code generated]
  modules by path ./node_modules/ 176 KiB
    modules by path ./node_modules/react/ 9.39 KiB 2 modules
    modules by path ./node_modules/react-dom/ 155 KiB 2 modules
    modules by path ./node_modules/large-number/ 2.27 KiB 2 mod
ules
    modules by path ./node_modules/scheduler/ 7.37 KiB 2 module
s
    ./node_modules/object-assign/index.js 2.17 KiB [built] [cod
e generated]
webpack 5.4.0 compiled successfully in 4342 ms
```

webpack 5

```
Hash: dba7cb324c27945f23e0
Version: webpack 4.31.0
Time: 4246ms
Built at: 11/09/2020 3:33:44 PM
                Asset       Size  Chunks             Chunk Names
       4_a7315f43.js  191 bytes       4  [emitted]
       5_fcfa0452.js  191 bytes       5  [emitted]
       6_5785d676.js  191 bytes       6  [emitted]
          index.html  293 bytes          [emitted]
    index_80a49f23.js  994 bytes       0  [emitted]  index
    logo_bd62f047.png   8.54 KiB          [emitted]
         search.html   3.12 KiB          [emitted]
        search1.html   3.13 KiB          [emitted]
```

首次构建

```
Hash: dba7cb324c27945f23e0
Version: webpack 4.31.0
Time: 2745ms
Built at: 11/09/2020 3:34:29 PM
                Asset       Size  Chunks             Chunk Names
       4_a7315f43.js  191 bytes       4  [emitted]
       5_fcfa0452.js  191 bytes       5  [emitted]
       6_5785d676.js  191 bytes       6  [emitted]
          index.html  293 bytes          [emitted]
    index_80a49f23.js  994 bytes       0  [emitted]  index
    logo_bd62f047.png   8.54 KiB          [emitted]
         search.html   3.12 KiB          [emitted]
        search1.html   3.13 KiB          [emitted]
 search1_7dbb282c.js    121 KiB       2  [emitted]  search1
```

二次构建

webpack 4

目 录

CONTENTS

01 | webpack 5 新特性解析

02 | bundle 和 bundless 的差异

03 | Vite 的构建原理

Vite 发布 2.0 版本



尤小右

4月26日 06:05 来自 微博国际版

过去一周捣鼓了一个去年就试验过但没完成的想法：

网页链接

Vite，一个基于浏览器原生 ES imports 的开发服务器。利用浏览器去解析 imports，在服务器端按需编译返回，完全跳过了打包这个概念，服务器随起随用。同时不仅有 Vue 文件支持，还搞定了热更新，而且热更新的速度不会随着模块增多而变慢。针对生产环境则可以把同一份代码用 rollup 打包。虽然现在还比较粗糙，但这个方向我觉得是有潜力的，做得好可以彻底解决改一行代码等半天热更新的问题。 收起全文

收藏 | 201 | 81 | 535

更早实践 bundless 的工具：Snowpack



真的可以不用打包吗?

浏览器对 ESM 的支持情况

Can I use javascript modules ? Settings

3 results found

JavaScript modules via script tag - LS

Loading JavaScript module scripts using `<script type="module">` Includes support for the `nomodule` attribute.

Usage: Global — % of all users ? — 93.07% + 0.32% = 93.39%

Current aligned | Usage relative | Date relative | Filtered | All

IE	Edge *	Firefox	Chrome	Safari	Opera	iOS Safari *	Opera Mini *	Android Browser *	Opera Mobile *	Chrome for Android	Firefox for Android	UC Browser for Android	Samsung Internet	QQ Browser	Baidu Browser	KaiOS Browser
	12-14															
	1 6 15	2-53	4-59	3.1-10	10-46	3.2-10.2										
	6 16-18	2 54-59	1 60	4 5 10.1	1 47	4 5 10.3							4-7.4			
6-10	79-85	60-81	61-85	11-13.1	48-71	11-13.7		2.1-4.4.4	12-12.1				8.2-11.2			
11	86	82	86	14	72	14	all	81	59	86	82	12.12	12.0	10.4	7.12	2.5
		83-84	87-89	TP												

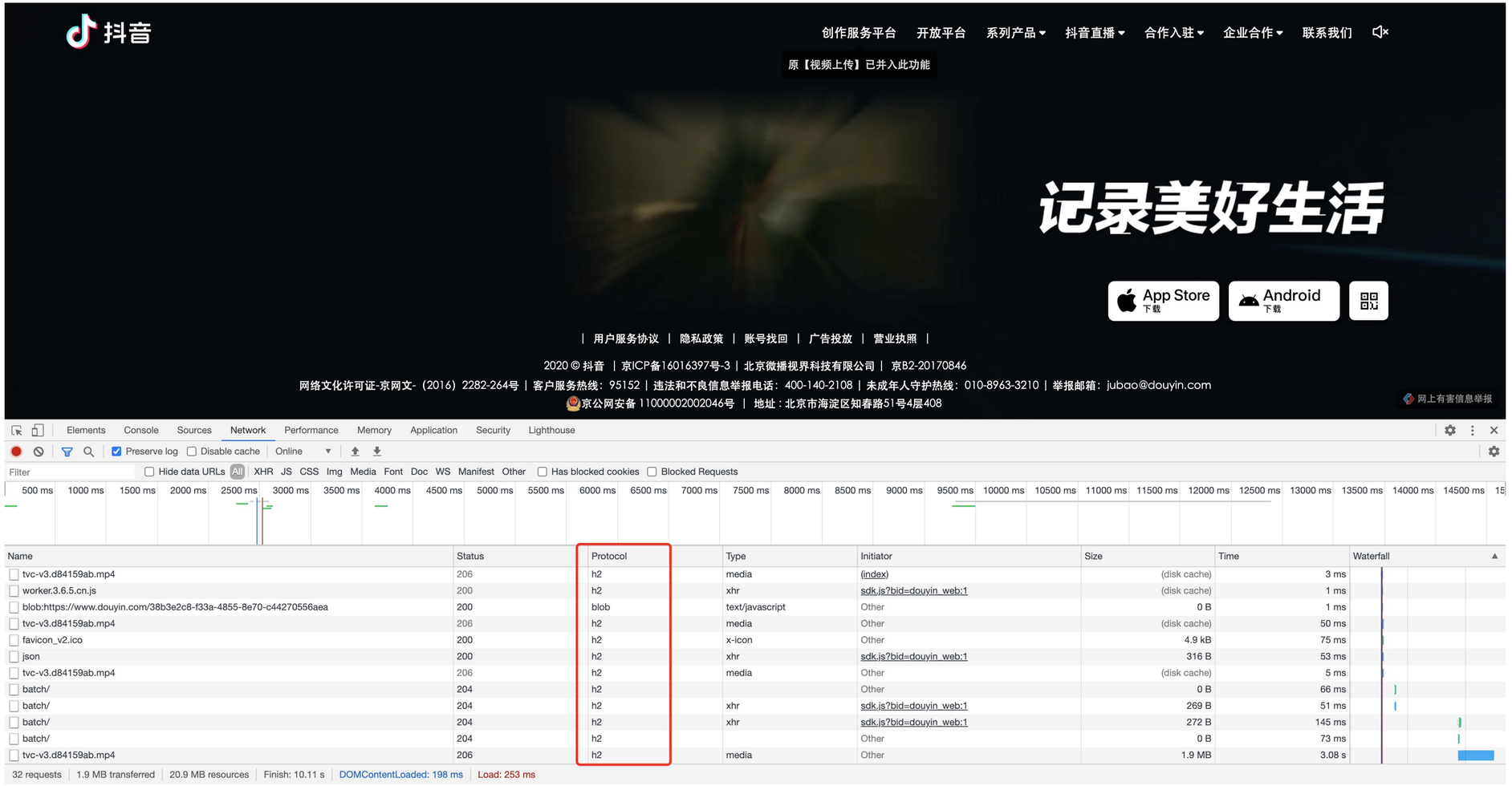
HTTP/1.1 的请求数限制

Browser	Connections per Domain	Max Connections
Chrome 81	6 [^note1]	256[^note2]
Edge 18	*same as Internet Explorer 11*	*same as Internet Explorer 11*
Firefox 68	9 [^note1] or 6 [^note3]	1000+[^note2]
Internet Explorer 11	12 [^note4]	1000+[^note2]
Safari 13	6 [^note1]	1000+[^note2]

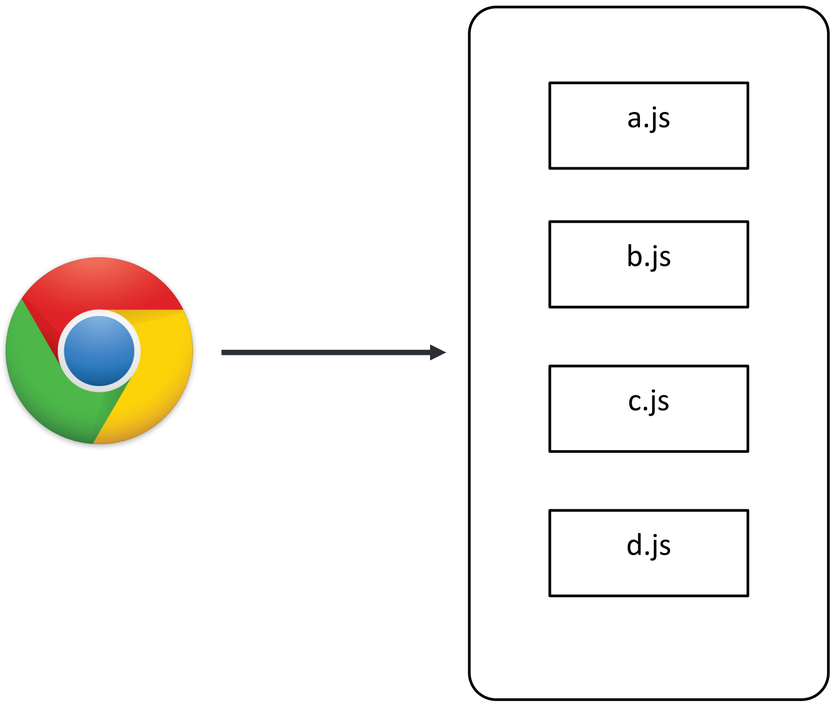
- [^ note1]: tested with 72 requests , 1 domain(127.0.0.1)
- [^ note2]: tested with 1002 requests, 6 requests per domain * 167 domains (127.0.0.*)
- [^ note3]: when called in async context, e.g. in callback of `setTimeout` , + `requestAnimationFrame` , `then` ...
- [^ note4]: of which the last 6 are follow-ups (2,4,6 available at 0.5s,1s,1.5s respectively)

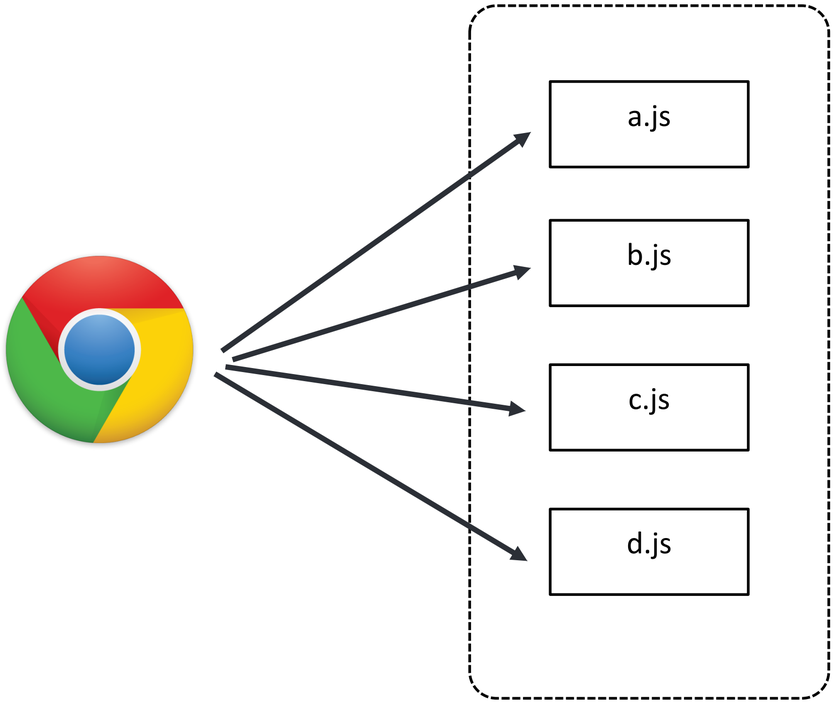
各浏览器在 HTTP/1.1 协议下的最大并行请求数限制

HTTP/2 越来越多的应用在主流网站



资源加载的差异

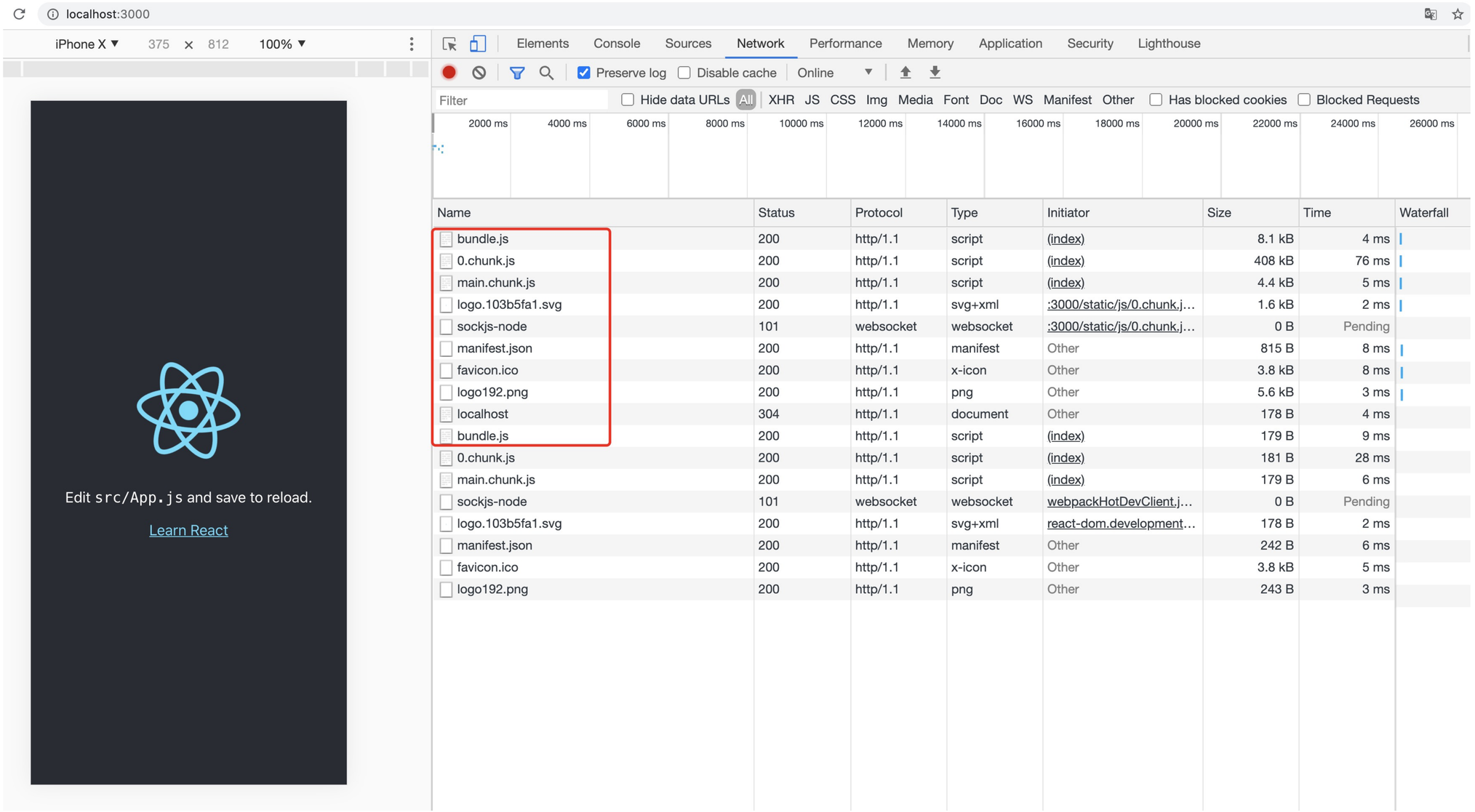




bundle

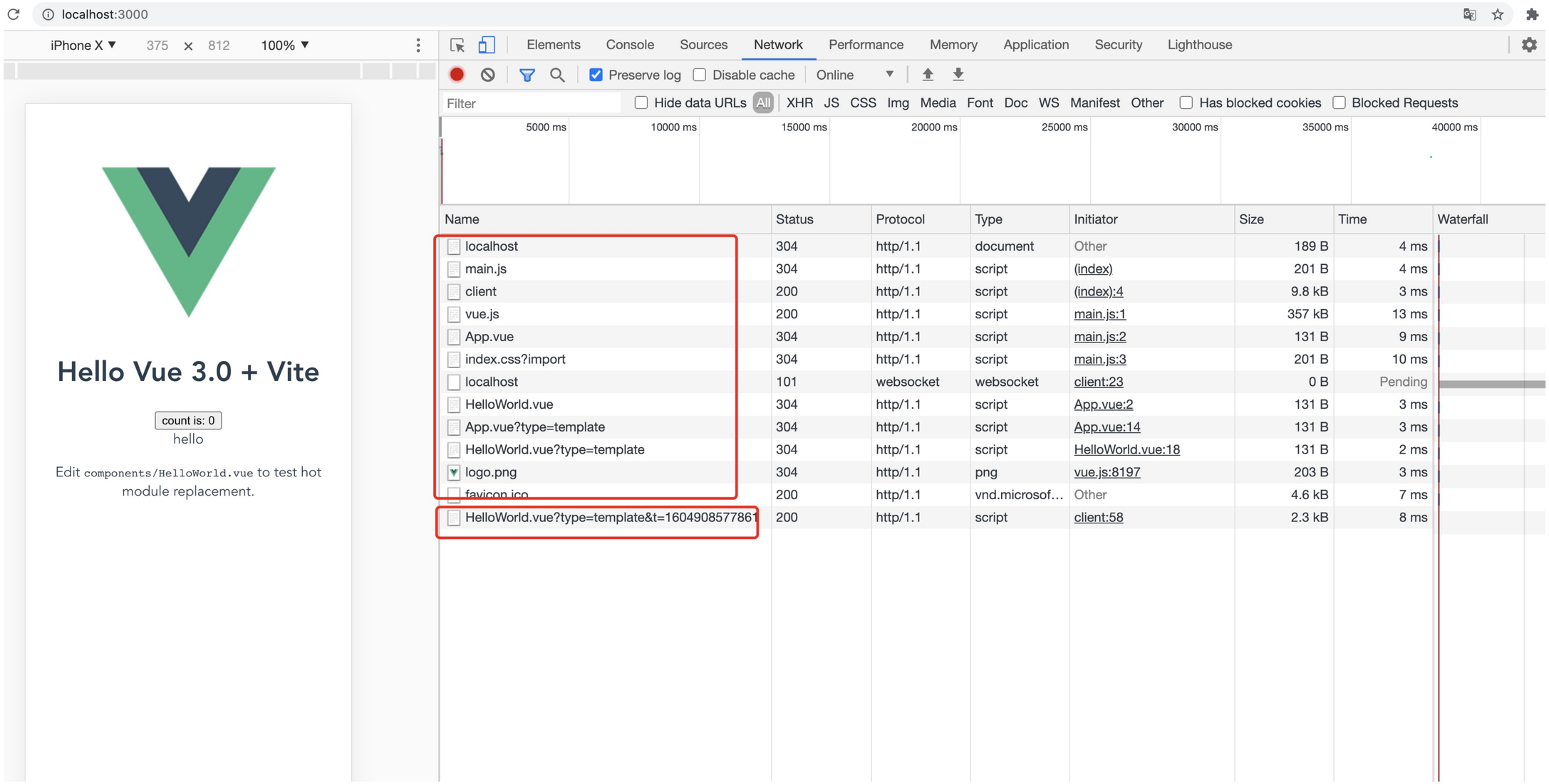
bundless

资源加载的差异 – bundle

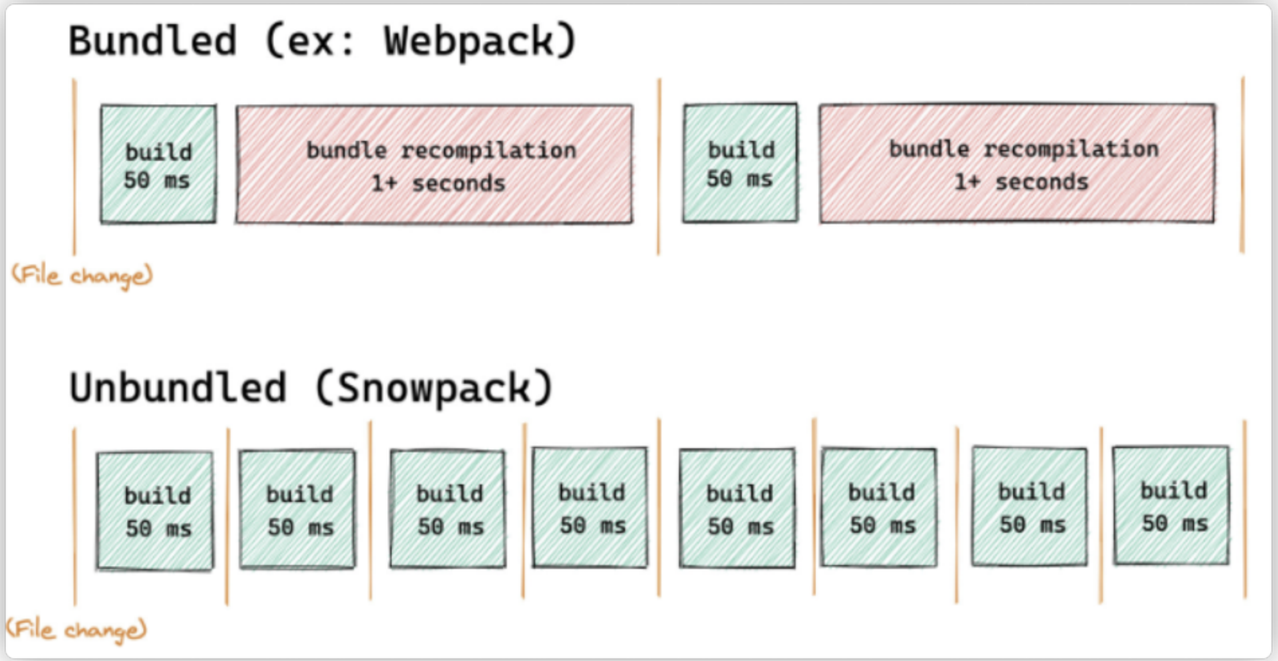


Name	Status	Protocol	Type	Initiator	Size	Time	Waterfall
bundle.js	200	http/1.1	script	(index)	8.1 kB	4 ms	
0.chunk.js	200	http/1.1	script	(index)	408 kB	76 ms	
main.chunk.js	200	http/1.1	script	(index)	4.4 kB	5 ms	
logo.103b5fa1.svg	200	http/1.1	svg+xml	:3000/static/js/0.chunk.j…	1.6 kB	2 ms	
sockjs-node	101	websocket	websocket	:3000/static/js/0.chunk.j…	0 B	Pending	
manifest.json	200	http/1.1	manifest	Other	815 B	8 ms	
favicon.ico	200	http/1.1	x-icon	Other	3.8 kB	8 ms	
logo192.png	200	http/1.1	png	Other	5.6 kB	3 ms	
localhost	304	http/1.1	document	Other	178 B	4 ms	
bundle.js	200	http/1.1	script	(index)	179 B	9 ms	
0.chunk.js	200	http/1.1	script	(index)	181 B	28 ms	
main.chunk.js	200	http/1.1	script	(index)	179 B	6 ms	
sockjs-node	101	websocket	websocket	webpackHotDevClient.j…	0 B	Pending	
logo.103b5fa1.svg	200	http/1.1	svg+xml	react-dom.development…	178 B	2 ms	
manifest.json	200	http/1.1	manifest	Other	242 B	6 ms	
favicon.ico	200	http/1.1	x-icon	Other	3.8 kB	5 ms	
logo192.png	200	http/1.1	png	Other	243 B	3 ms	

资源加载的差异 – bundless



打包速度的对比



bundless 的冷启动时间大大缩短

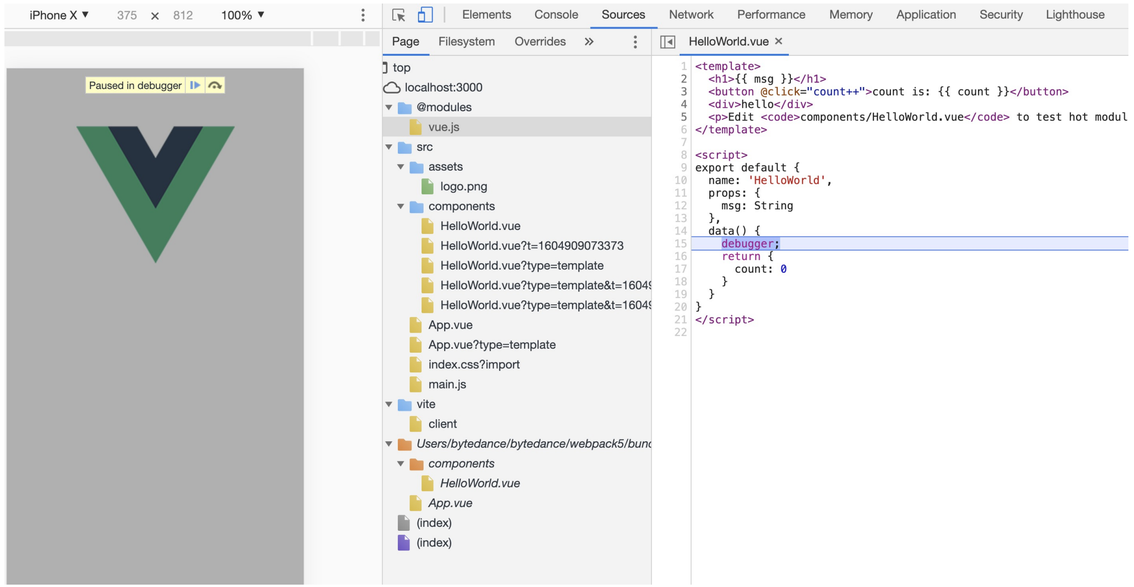
bundless 的 HMR 速度不受整个项目体积影响

bundless 的单文件粒度的缓存更优

开发体验的对比

以 webpack 里面的 sourcemap 类型举例

devtool	首次构建	二次构建	是否适合生产环境	可以定位的代码
(none)	+++	+++	yes	最终输出的代码
eval	+++	+++	no	webpack生成的代码(一个个的模块)
cheap-eval-source-map	+	++	no	经过loader转换后的代码(只能看到行)
cheap-module-eval-source-map	o	++	no	源代码(只能看到行)
eval-source-map	--	+	no	源代码
cheap-source-map	+	o	yes	经过loader转换后的代码(只能看到行)
cheap-module-source-map	o	-	yes	源代码(只能看到行)
inline-cheap-source-map	+	o	no	经过loader转换后的代码(只能看到行)
inline-cheap-module-source-map	o	-	no	源代码(只能看到行)
source-map	--	--	yes	源代码
inline-source-map	--	--	no	源代码
hidden-source-map	--	--	yes	源代码



bundle 依赖 sourcemap

bundless 调试简单

目 录

CONTENTS

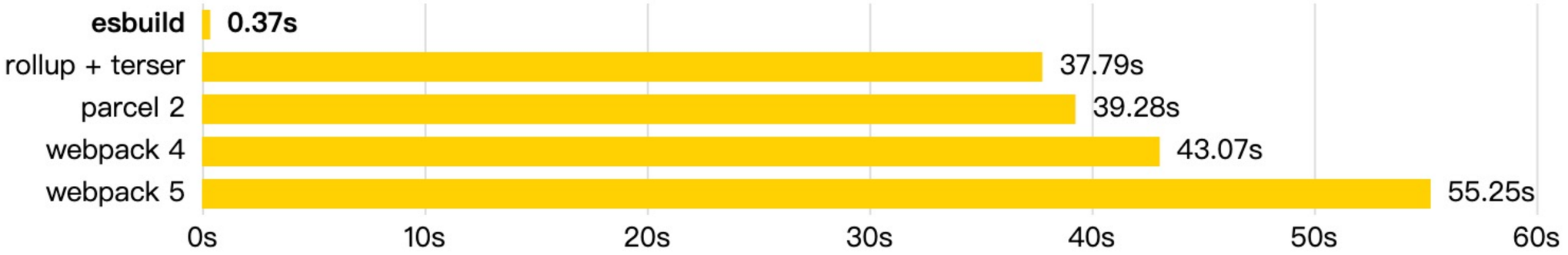
01 | webpack 5 新特性解析

02 | bundle 和 bundless 的差异

03 | Vite 的构建原理

Vite 构建速度快的原因

预构建使用 ESBuild（冷启动快的原因）：**https://github.com/evanw/esbuild**



Vite 打包流程 – 创建构建服务

文件路径：src/node/server/index.ts

```ts
export function createServer(config: ServerConfig): Server {
  // 创建 KOA Server
  const app = new Koa<State, Context>()
  const server = resolveServer(config, app.callback())
  // 使用 chokidar 监听文件变化，用于 HMR
  const watcher = chokidar.watch(root, {
    ignored: [/node_modules/, /\.git/],
    // #610
    awaitWriteFinish: {
      stabilityThreshold: 100,
      pollInterval: 10
    }
  }) as HMRWatcher
  // 模块解析器
  const resolver = createResolver(root, resolvers, alias, assetsInclude)

  const context: ServerPluginContext = {}

  // 各种中间件
  const resolvedPlugins = [

  ]
  resolvedPlugins.forEach((m) => m && m(context))

  const listen = server.listen.bind(server)
  server.listen = (async (port: number, ...args: any[]) => {
    if (optimizeDeps.auto !== false) {
      // 执行预优化
      await require('../optimizer').optimizeDeps(config)
    }
    return listen(port, ...args)
  }) as any

  server.once('listening', () => {
    context.port = (server.address() as AddressInfo).port
  })

  return server
}
```

创建 KOA Server

↓

使用 chokidar 监听文件变化

↓

模块解析

↓

增加各种中间件

↓

启动服务

Vite 打包流程 — 静态文件托管服务

文件路径：src/node/server/index.ts

```
export function createServer(config: ServerConfig): Server {
  // 创建 KOA Server
  const resolvedPlugins = [
    sourceMapPlugin,
    moduleRewritePlugin,
    htmlRewritePlugin,
    // 用户自定义的插件
    ...toArray(configureServer),
    envPlugin,
    moduleResolvePlugin,
    proxyPlugin,
    clientPlugin,
    hmrPlugin,
    ...(transforms.length || Object.keys(vueCustomBlockTransforms).length
      ? [
          createServerTransformPlugin(
            transforms,
            vueCustomBlockTransforms,
            resolver
          )
        ]
      : []),
    vuePlugin,
    cssPlugin,
    enableEsbuild ? esbuildPlugin : null,
    jsonPlugin,
    assetPathPlugin,
    webWorkerPlugin,
    wasmPlugin,
    serveStaticPlugin
  ]
  // 应用插件
  resolvedPlugins.forEach((m) => m && m(context))
}
```

Vite 会利用 serveStaticPlugin 将整个项目根目录、Public 目录设置为静态目录。

serveStaticPlugin 利用 koa-etag 中间件打 etag

devServer 具备静态文件服务功能

Vite 打包流程 — 重写模块路径

文件路径：src/node/server/index.ts

```ts
export function createServer(config: ServerConfig): Server {
  // 创建 KOA Server
  const resolvedPlugins = [
    sourceMapPlugin,
    moduleRewritePlugin,
    htmlRewritePlugin,
    // 用户自定义的插件
    ...toArray(configureServer),
    envPlugin,
    moduleResolvePlugin,
    proxyPlugin,
    clientPlugin,
    hmrPlugin,
    ...(transforms.length || Object.keys(vueCustomBlockTransforms).length
      ? [
          createServerTransformPlugin(
            transforms,
            vueCustomBlockTransforms,
            resolver
          )
        ]
      : []),
    vuePlugin,
    cssPlugin,
    enableEsbuild ? esbuildPlugin : null,
    jsonPlugin,
    assetPathPlugin,
    webWorkerPlugin,
    wasmPlugin,
    serveStaticPlugin
  ]
  // 应用插件
  resolvedPlugins.forEach((m) => m && m(context))
}
```

处理裸导入（bare import）的模块路径，eg:

import react from 'react';

浏览器（只有相对路径和绝对路径）**无法识别，因此需要重写模块路径**

Vite 打包流程 — 重写模块路径

文件路径：src/node/server/serverPluginModuleRewrite.ts

```ts
export const resolveImport = (
  root: string,
  importer: string,
  id: string,
  resolver: InternalResolver,
  timestamp?: string
): string => {
  id = resolver.alias(id) || id

  if (bareImportRE.test(id)) {
    // directly resolve bare module names to its entry path so that relative
    // imports from it (including source map urls) can work correctly
    id = `/@modules/${resolveBareModuleRequest(root, id, importer, resolver)}`
  } else {
    // 将相对路径转换成绝对路径
    let { pathname, query } = resolver.resolveRelativeRequest(importer, id)

    // 补全文件扩展名和经常省略的 index.xxx
    pathname = resolver.normalizePublicPath(pathname)

    // 非 js 类型的文件，加上一个 import 的 query 参数
    if (!query && path.extname(pathname) && !jsSrcRE.test(pathname)) {
      query += `?import`
    }

    id = pathname + query
  }

  // 4. force re-fetch dirty imports by appending timestamp
  if (timestamp) {
    const dirtyFiles = hmrDirtyFilesMap.get(timestamp)
    const cleanId = cleanUrl(id)
    // only rewrite if:
    if (dirtyFiles && dirtyFiles.has(cleanId)) {
      // 1. this is a marked dirty file (in the import chain of the changed file)
      id += `${id.includes(`?`) ? `&` : `?`}t=${timestamp}`
    } else if (latestVersionsMap.has(cleanId)) {
      // 2. this file was previously hot-updated and has an updated version
      id += `${id.includes(`?`) ? `&` : `?`}t=${latestVersionsMap.get(cleanId)}`
    }
  }
  return id
}
```

对于 bare import，把模块名替换为这个模块的 entry path，并在 path 的开头补上一个 /@modules 的标识符。

如：import React from "/@modules/@pika/react/source.development.js"

相对路径转绝对路径，方便浏览器请求。

补齐文件扩展名和经常被省略的 index.xxx

如：import a from ’/src/a/index.js’

给非 js 类型（ js 类型：如 js(x)/ts(x)/vue ）的文件地址加上一个叫 “import” 的 query 参数。

如：import ‘/src/style.css?import’

给 hmr 相关的请求地址添加时间戳，避免缓存。

如：import ‘/src/App.jsx?t=1599124870589’

Vite 打包流程 — 静态资源打包策略

浏览器不支持 JS 中直接写 import CSS、图片、JSON 等语法。

回顾一下 webpack loader 的处理策略：

- CSS：转换成 js 的模块，执行模块会在 DOM 中创建 <style> 标签并且插入 CSS 内容
- 图片：转换成图片路径
- JSON: 转化成 js 模块，default export = json

Vite 打包流程 — vue 脚本打包策略

```
const compilerSfc = require('@vue/compiler-sfc');

if (url.includes('.vue')) {
  const p = path.resolve(__dirname, url.slice(1));
  const { descriptor } = compilerSfc.parse(fs.readFileSync(p, 'utf-8'));
  if (!query.type) {
    ctx.type = 'application/javascript';
    ctx.body = `
      const __script = ${descriptor.script.content.replace('export default ', '')}
      ${descriptor.styles.length ? `import "${url}?type=style"` : ''}
      import { render as __render } from "${url}?type=template"
      __script.render = __render;
      export default __script;
    `;
  }
}
```

获取 Script 内容

如果有 style 就发送请求获取 style 的部分

发送请求获取 template 的部分

进行渲染

Vite 打包流程 — CSS 资源打包策略

```ts
export const cssPlugin: ServerPlugin = ({ root, app, watcher, resolver }) => {
  app.use(async (ctx, next) => {
    await next()
    // 判断是否是 .css 文件请求
    if (
      isCSSRequest(ctx.path) &&
      // note ctx.body could be null if upstream set status to 304
      ctx.body
    ) {
      const id = JSON.stringify(hash_sum(ctx.path))
      // 判断是否为 js 中 import 的 css
      if (isImportRequest(ctx)) {
        // 进行 css 的编译处理
        const { css, modules } = await processCss(root, ctx)
        ctx.type = 'js'

        // 基于 css 源码包装成 js 模块
        ctx.body = codegenCss(id, css, modules)
      }
    }
  })
}
```

判断是否是 .css 文件的请求

判断是否为 JS 中 import 的 CSS

进行 Sass/Less 的编译处理

基于 CSS 的源码包装成 JS 的模块

Vite 打包流程 — CSS 资源打包策略

```ts
export function codegenCss(
  id: string,
  css: string,
  modules?: Record<string, string>
): string {
  let code =
    `import { updateStyle } from "${clientPublicPath}"\n` +
    `const css = ${JSON.stringify(css)}\n` +
    `updateStyle(${JSON.stringify(id)}, css)\n`
  if (modules) {
    code += dataToEsm(modules, { namedExports: true })
  } else {
    code += `export default css`
  }
  return code
}
```

将 CSS 源码加工成 JS 模块

updateStyle 会通过 CSS Style Shee API 后者直接创建 <style> 标签

将 CSS 代码插入到 DOM 中

如果开启了 CSS Module，则直接导出成一个对象，否则导出 CSS 代码

Vite 打包流程 —模板打包策略

@vue/compiler-dom 编译 template，然后返回给浏览器

```
const compilerDom = require('@vue/compiler-dom');

...
if (query.type === 'template') {
  const template = descriptor.template;
  // 在服务端编译 template 并且返回
  const render = compilerDom.compile(template.content, {
    mode: 'module',
  }).code;
  ctx.type = 'application/javascript';
  ctx.body = render;
}
```